

Algoritmos — V4: Aprendendo a Executar e Calcular do Zero

Em algoritmos, calcular significa acompanhar os valores das variáveis até descobrir exatamente o que acontece.

0. Método principal

ENTRADA → PROCESSAMENTO → DECISÃO/REPETIÇÃO → SAÍDA

Em loops, transforme a execução em uma tabela.

1. Estado inicial

Exemplo:

```
int x = 10;  
int y = 3;
```

Estado:

Variável	Valor
x	10
y	3

Tudo começa aqui.

2. Atribuição

Considere:

```
x = x + 2;
```

Se x=10:

10+2=12

Novo valor:

x=12.

Regra

LEIA = COMO “RECEBA”.

Não confunda com uma igualdade matemática.

3. Operações

Se $a=10$ e $b=3$:

$a+b=13$

$a-b=7$

$a*b=30$

$a/b=3$ quando ambos são int

$a\%b=1$

O operador $\%$ representa o resto da divisão.

4. Comparações

Se $x=8$:

$x>5 \rightarrow \text{true}$

$x<5 \rightarrow \text{false}$

$x==8 \rightarrow \text{true}$

$x!=8 \rightarrow \text{false}$

Atenção

$=$ atribui.

$==$ compara.

5. if e else

Exemplo:

```
int x = 7;
```

```
if (x > 5) {  
    System.out.println("A");  
} else {  
    System.out.println("B");  
}
```

Calcule:

$7>5 \rightarrow \text{true}$

Então entra no if.

Saída:

A

Regra

PRIMEIRO CALCULE A CONDIÇÃO. DEPOIS ESCOLHA O BLOCO.

6. && e ||

&& só é verdadeiro quando as duas condições são verdadeiras.

|| é verdadeiro quando pelo menos uma é verdadeira.

Exemplo:

x=10

x>=5 → true

x<=12 → true

Logo:

true && true → true

7. for: a ordem correta

Exemplo:

```
for (int i = 0; i < 5; i++) {  
    System.out.println(i);  
}
```

A ordem é:

1. inicializa;
2. testa condição;
3. executa corpo;
4. atualiza;
5. volta para a condição.

Tabela:

Iteração	i	i<5	Saída
1	0	true	0
2	1	true	1
3	2	true	2
4	3	true	3
5	4	true	4
6	5	false	para

Resultado:

0,1,2,3,4.

8. **i++**

i++ significa:

i = i + 1

Se i=3, depois do incremento:

i=4.

Não significa +2.

9. **i += 2**

Aqui:

i=i+2

Começando em 0:

0,2,4,6,8...

Memória

++ = +1

+=2 = +2

10. **++i x i++**

Se:

```
int i = 5;  
System.out.println(i++);
```

usa primeiro:

saída 5

depois i=6.

Agora:

```
int i = 5;  
System.out.println(++i);
```

primeiro:

5+1=6

depois usa.

saída 6.

11. Filtro de pares

Exemplo:

```
for (int i = 0; i <= 10; i++) {  
    if (i % 2 == 0) {  
        System.out.println(i);  
    }  
}
```

O for produz:

0,1,2,3,4,5,6,7,8,9,10

O if seleciona:

0,2,4,6,8,10

Pegadinha

i++ produz todos os números; o if filtra os pares.

12. Soma acumulada

Exemplo:

```
int soma = 0;  
  
for (int i = 1; i <= 4; i++) {  
    soma = soma + i;  
}
```

Tabela:

i	soma antes	conta	soma depois
1	0	0+1	1
2	1	1+2	3
3	3	3+3	6
4	6	6+4	10

Resultado:

10.

13. Contador

Exemplo:

```
int cont = 0;  
  
for (int i = 1; i <= 6; i++) {
```

```

        if (i % 2 == 0) {
            cont++;
        }
    }
}

```

Pares:

2,4,6

cont aumenta três vezes.

Resultado:

cont=3.

14. while

Exemplo:

```

int i = 0;

while (i < 3) {
    System.out.println(i);
    i++;
}

```

Tabela:

i	condição	saída	novo i
0	true	0	1
1	true	1	2
2	true	2	3
3	false	—	para

Resultado:

0,1,2.

15. Teste de mesa

Para cada variável importante, crie uma coluna.

Exemplo:

i	soma	condição
0	0	true
1	0	true
2	2	true

A tabela não precisa ser bonita. Precisa registrar o estado.

16. Debugger

Breakpoint → pausa.

Step Over → executa a próxima instrução sem entrar no método.

Step Into → entra no método.

Step Out → volta para quem chamou.

Evaluate Expression → calcula uma expressão durante a pausa.

Memória:

PAUSA → AVANÇA → ENTRA → SAI → CALCULA

17. Erros

Sintaxe

Não compila.

Runtime

Começa a executar e falha durante a execução.

Lógica

Executa, mas o resultado está errado.

Pergunte:

nem compilou?

quebrou executando?

terminou errado?

18. Javadoc

@param = entrada.

@return = saída.

@throws = possíveis exceções.

Pense no método como:

ENTRADA → PROCESSAMENTO → SAÍDA

19. Exemplo completo

Considere:

```
int valor = 10;
int soma = 0;

for (int i = 0; i <= valor; i++) {
    if (i % 2 == 0) {
        soma += i;
    }
}
```

Passo 1

i percorre:

0,1,2,3,4,5,6,7,8,9,10

Passo 2

O filtro mantém:

0,2,4,6,8,10

Passo 3

Some:

0+2+4+6+8+10=30

Resultado:

soma=30.

Erro clássico

Achar que `i++` já cria a sequência de pares.

Não.

O incremento é `+1`.

20. Loops aninhados

Exemplo:

```
for (int i = 1; i <= 2; i++) {
    for (int j = 1; j <= 3; j++) {
        System.out.println(i + "," + j);
    }
}
```

Para `i=1`, `j` faz 1,2,3.

Para `i=2`, `j` faz 1,2,3 novamente.

Total:

$2 \times 3 = 6$ execuções internas.

Regra

O LOOP INTERNO RECOMEÇA A CADA NOVA RODADA DO EXTERNO.

21. Como resolver no papel

1. escreva valores iniciais;
2. execute linha por linha;
3. em if, calcule a condição;
4. em loop, monte uma tabela;
5. ao final, leia a saída.

Não pule uma atualização só porque parece óbvia.

22. DEU MERDA

i++ confundido com +2

ERRO: pensar que ++ significa saltar de dois.

CONSEQUÊNCIA: sequência errada.

REGRA: $i++ = i+1$.

esquecer atualização

ERRO: acompanhar só a condição.

CONSEQUÊNCIA: próximo estado incorreto.

REGRA: loop = condição + corpo + atualização.

23. Resumo

= atribui

== compara

% = resto

if = calcule condição

i++ = +1

++i = incrementa antes de usar

i+=2 = +2

for = inicializa → testa → executa → atualiza

while = testa → executa → atualiza

teste de mesa = registre o estado

loop aninhado = interno reinicia

24. Checklist

- ☐ Sei atribuição.
- ☐ Sei operadores.
- ☐ Sei %.
- ☐ Sei if/else.
- ☐ Sei && e ||.
- ☐ Sei for.
- ☐ Sei while.
- ☐ Sei i++, ++i e +=2.
- ☐ Sei contador e acumulador.
- ☐ Sei teste de mesa.
- ☐ Sei loop aninhado.
- ☐ Sei classificar erros.

Fórmula de sobrevivência: escreva cada mudança de variável. Quando o estado está no papel, o algoritmo deixa de parecer uma sequência mágica.